J.N.T.U.H UNIVERSITY COLLEGE OF ENGINEERING HYDERABAD

(Autonomous)

KUKATPALLY, HYDERABAD - 500 085



Certificate

Certified that this is the bonafide record of the practical work done during the academic year _____ by

Name _____

Roll Number _____ Class _____

In the Laboratory of _____

of the Department of _____

Signature of the Staff Member ***Signature of the Head of the Department***

Date of Examination_____

Signature of the Examiner/s

Internal Examiner ***External Examiner*** q

J.N.T.U.H UNIVERSITY COLLEGE OF ENGINEERING HYDERABAD

(Autonomous)

KUKATPALLY, HYDERABAD - 500 085

Name _____ ***Roll Number*** _____

Class _____ ***Year*** _____

Laboratory_____

List of Experiments

S.No.	Name of the Experiment	Date of Experiment	Page Number	Marks	Remarks
1	Write a C program to recognise keywords of flow control statements and an identifier of C language.		1		
2	Write a C program to calculate first function for the grammar: E → E + T E → T T → T * F T → F F → (E)/ id		2		
3	Write a C program to implement Predictive Parsing for the grammar: E → TQ Q → +TQ/€ T → FR R → *FR/€ F → (E) / id		6		
4	Write a C program for Stack implementation of SLR parsing for the grammar: E → E + T E → T T → T * F T → F F → (E) F → id		12		
5	Write a C program for Stack implementation of SLR parsing for the grammar: S → AB		16		

	A → a / € B → b / €				
6	Write recursive descent parser for the grammar: E → TE' E' → +TE' / € T → FT' T' → *FT'/ € F → (E) / id		19		
7	Write recursive descent parser for the grammar: S → (L) / a L → SL' L' → ,SL' / €		22		
8	Implement any Static storage allocation strategy.		24		
9	Implement any Stack-based storage allocation strategy		25		
10.	Implement any Heap-based storage allocation strategy.		28		
11.	Implement Three Address Code using syntax Tree.		32		
12.	Implement Three Address Code using Directed Acyclic Graph.		35		
13.	Write a Lex specification to recognize +ve integers, reals and -ve integers, reals.		39		
14.	Write a Lex specification for converting real numbers to integers.		40		
15.	Write a Lex specification to print the number of days in a month using a procedure		41		
16.	Write a Lex specification to print a number in between 0-100 in words.		42		
17.	Write a Lex specification to print a number in between 0-1000 in words		44		
18.	Write a Lex specification to retrieve comments.		46		
19.	Write a Lex specification to design a lexical analyzer that recognizes identifiers and keywords of flow control statements of C language.		47		

1. Write a C program to recognise keywords of flow control statements and an identifier of C language.

Program:

```
#include <stdio.h>
int main()
{
  char str[50];
  const char *keywords[] = {
    "if", "else", "while", "do", "switch",
    "case", "default", "continue", "return",
    "goto", "for", "break"};
  int n = sizeof(keywords) / sizeof(keywords[0]);
  int found = 0;
  printf("Enter a word: ");
  scanf("%49s", str);
  for (int i = 0; i < n; i++)
  {
    int j = 0;
    while (str[j] == keywords[i][j] && str[j] != '\0')
      j++;
    if (str[j] == '\0' && keywords[i][j] == '\0')
    {
      found = 1;
      break;
    }
  }
  if (found) {
    printf("%s is a flow control keyword\n", str);
  }else{
    if ((str[0] >= 'A' && str[0] <= 'Z') || (str[0] >= 'a' && str[0] <= 'z') || (str[0] == '_')) {
```

```
        found = 1;
        for (int i = 1; str[i] != '\0'; i++)
        {
          if (!((str[i] >= 'A' && str[i] <= 'Z') ||
              (str[i] >= 'a' && str[i] <= 'z') ||
              (str[i] >= '0' && str[i] <= '9') ||
              (str[i] == '_'))) {
            found = 0;
            break;
          }
        }
        if (found)
          printf("\n%s is a valid identifier", str);
        else
          printf("\n%s is not a valid identifier", str);
      }else{
        printf("\n%s is not a valid identifier", str);
      }
    }
    return 0;
}
```

OUTPUT:

```
PS C:\MinGW\bin> & "C:\Users\Tejasree Nethula\OneDrive\Desktop\cd-lab\first.exe"
Enter a word: for
for is a flow control keyword
PS C:\MinGW\bin> & "C:\Users\Tejasree Nethula\OneDrive\Desktop\cd-lab\first.exe"
Enter a word: num23

num23 is a valid identifier
```

2. Write a C program to calculate first function for the grammar:

E → E + T
E → T

T → T * F
T → F
F → (E)
F → id

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char lhs[20];
char rhs[20][20][10];
int n, count[20];
char first[26][20][10];
int fcount[26];
int add(int nt, char *s)
{
   for (int i = 0; i < fcount[nt]; i++)
      if (strcmp(first[nt][i], s) == 0)
         return 0;
   strcpy(first[nt][fcount[nt]++], s);
   return 1;
}
int main()
{
   printf("Enter number of productions: ");
   scanf("%d", &n);

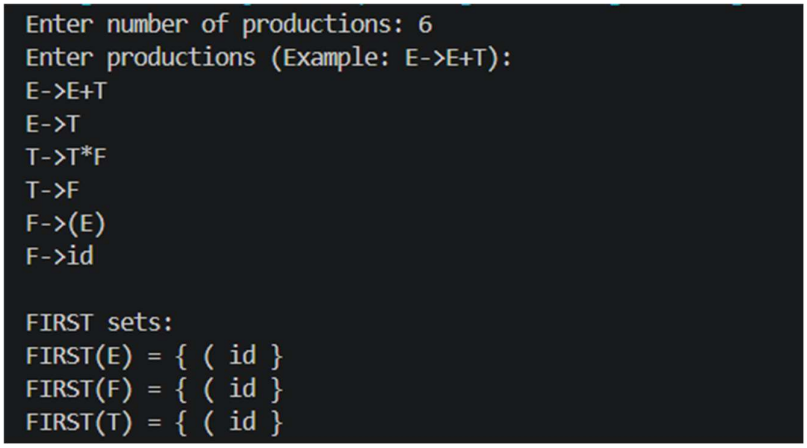
   printf("Enter productions (Example: E->E+T):\n");
   for (int i = 0; i < n; i++)
   {
      char line[50];
```

```
    scanf("%s", line);
    int nt = line[0] - 'A';
    int j = 3, k = 0;
    lhs[i] = line[0];
    while (line[j])
    {
        if (isupper(line[j]))
        {
            rhs[i][k][0] = line[j++];
            rhs[i][k++][1] = '\0';
        }
        else if (islower(line[j]))
        {
            int p = 0;
            while (islower(line[j]))
                rhs[i][k][p++] = line[j++];
            rhs[i][k++][p] = '\0';
        }
        else
        {
            rhs[i][k][0] = line[j++];
            rhs[i][k++][1] = '\0';
        }
    }
    count[i] = k;
}
int changed;
do
{
    changed = 0;
```

```
    for (int i = 0; i < n; i++)
    {
        int nt = lhs[i] - 'A';
        char *s = rhs[i][0];
        if (isupper(s[0]))
        {
            int x = s[0] - 'A';
            for (int j = 0; j < fcount[x]; j++)
                changed |= add(nt, first[x][j]);
        }
        else
        {
            changed |= add(nt, s);
        }
    }

  } while (changed);
  printf("\nFIRST sets:\n");
  for (int i = 0; i < 26; i++)
  {
    if (fcount[i])
    {
        printf("FIRST(%c) = { ", i + 'A');
        for (int j = 0; j < fcount[i]; j++)
            printf("%s ", first[i][j]);
        printf("}\n");
    }
  }
  return 0;
}
```

OUTPUT:



3. Write a C program to implement Predictive Parsing for the grammar:

E → TQ
Q → +TQ/€
T → FR
R → *FR/€
F → (E) / id

Program:

```
#include <stdio.h>
#include <string.h>
char stack[100];
char input[100];
int top = -1, ip = 0;
void push(char c)
{
   stack[++top] = c;
}
char pop()
{
   return stack[top--];
}
```

```
char peek()
{
   return stack[top];
}
void firstfollow()
{
   printf("First(E) = { (, id }\n");
   printf("First(Q) = { +, e }\n");
   printf("First(T) = { (, id }\n");
   printf("First(R) = { *, e }\n");
   printf("First(F) = { (, id }\n");
   printf("Follow(E) = { ), $ }\n");
   printf("Follow(Q) = { ), $ }\n");
   printf("Follow(T) = { +, ), $ }\n");
   printf("Follow(R) = { +, ), $ }\n");
   printf("Follow(F) = { *, +, ), $ }\n");
}
int col(char ch)
{
   if (ch == 'i' && input[ip + 1] == 'd')
      return 0;
   switch (ch)
   {
   case '+':
      return 1;
   case '*':
      return 2;
   case '(':
```

```
      return 3;
   case ')':
      return 4;
   case '$':
      return 5;
   }
   return -1;
}
int row(char ch)
{
   switch (ch)
   {
   case 'E':
      return 0;
   case 'Q':
      return 1;
   case 'T':
      return 2;
   case 'R':
      return 3;
   case 'F':
      return 4;
   }

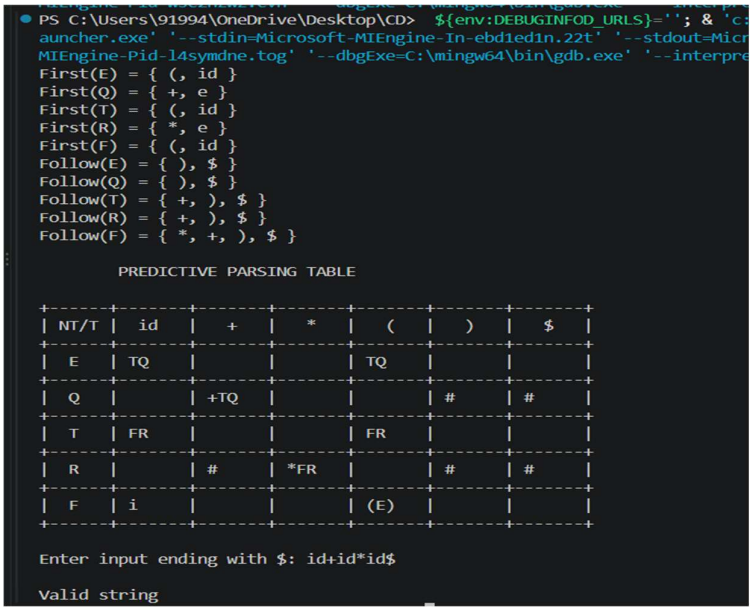
   return -1;
}
char *table[5][6] =
   {
```

```
    {"TQ", "", "", "TQ", "", ""},
    {"", "+TQ", "", "", "#", "#"},
    {"FR", "", "", "FR", "", ""},
    {"", "#", "*FR", "", "#", "#"},
    {"i", "", "", "(E)", "", ""}};
void printtable()
{
  int i, j;
  printf("\n\tPREDICTIVE PARSING TABLE\n\n");
  printf("+------+-------+-------+-------+-------+-------+-------+\n");
  printf("| NT/T |  id   |   +   |   *   |   (   |   )   |   $   |\n");
  printf("+------+-------+-------+-------+-------+-------+-------+\n");
  char nonterminals[] = {'E', 'Q', 'T', 'R', 'F'};
  for (i = 0; i < 5; i++)
  {
    printf("|  %c   ", nonterminals[i]);
    for (j = 0; j < 6; j++)
    {
      if (table[i][j][0] == '\0')
        printf("|       ");
      else
        printf("| %-5s ", table[i][j]);
    }
    printf("|\n");
    printf("+------+-------+-------+-------+-------+-------+-------+\n");
  }
}
int main()
```

```
{
   firstfollow();
   printtable();
   printf("\nEnter input ending with $: ");
   scanf("%99s", input);
   push('$');
   push('E');
   while (!(peek() == '$' && input[ip] == '$'))
   {
      char x = peek();
      char a = input[ip];
      if (x == 'i'){
         if (a == 'i' && input[ip + 1] == 'd'){
            pop();
            ip = ip + 2;
         }else{
            printf("\nInvalid string\n");
            return 0;
         }
      }else if (x == a){
         pop();
         ip++;
      }else if (x >= 'A' && x <= 'Z'){
         int r= row(x);
         int c = col(a);
         if (r == -1 || c == -1 || table[r][c][0] == '\0'){
            printf("\nInvalid string\n");
            return 0;
```

```
        }
        pop();
        if (table[r][c][0] != '#'){
           int len = strlen(table[r][c]);
           for (int j = len - 1; j >= 0; j--)
              push(table[r][c][j]);
        }
     }else{
        printf("\nInvalid string\n");
        return 0;
     }
  }
  if (input[ip] == '$')
     printf("\nValid string\n");
  else
     printf("\nInvalid string\n");
  return 0;
}
```

OUTPUT:



4. Write a C program for Stack implementation of SLR parsing for the grammar:

E → E + T
E → T
T → T * F
T → F
F → (E)
F → id

Program:

```
#include <stdio.h>
#include <string.h>
int a[12][6] = {
  {5, 0, 0, 4, 0, 0},
  {0, 6, 0, 0, 0, 999},
  {0, -2, 7, 0, -2, -2},
  {0, -4, -4, 0, -4, -4},
  {5, 0, 0, 4, 0, 0},
  {0, -6, -6, 0, -6, -6},
  {5, 0, 0, 4, 0, 0},
  {5, 0, 0, 4, 0, 0},
  {0, 6, 0, 0, 11, 0},
  {0, -1, 7, 0, -1, -1},
  {0, -3, -3, 0, -3, -3},
  {0, -5, -5, 0, -5, -5}};
int g[12][3] = {
  {1, 2, 3},
  {-1, -1, -1},
  {-1, -1, -1},
  {-1, -1, -1},
  {8, 2, 3},
  {-1, -1, -1},
  {-1, 9, 3},
```

```
    {-1, -1, 10},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1}};
char *p[] = {"", "E+T", "T", "T*F", "F", "(E)", "id"};
char lhs[] = {' ', 'E', 'E', 'T', 'T', 'F', 'F'};
int len[] = {0, 3, 1, 3, 1, 3, 1};
int main()
{
    char s[100], stack[100];
    int st[100], top = 0, i = 0, c;
    printf("Enter string: ");
    scanf("%98s", s);
    strcat(s, "$");
    stack[0] = '$';
    st[0] = 0;
    while (1){
        char ch = s[i];
        switch (ch){
        case 'i':
            if (s[i + 1] == 'd')
                c = 0;
            else
            {
                printf("Invalid input\n");
                return 0;
            }
            break;
```

```
case '+':
   c = 1;
   break;
case '*':
   c = 2;
   break;
case '(':
   c = 3;
   break;
case ')':
   c = 4;
   break;
case '$':
   c = 5;
   break;
default:
   printf("Invalid input\n");
   return 0;
}
int x = a[st[top]][c];
if (x == 999){
   printf("String Accepted\n");
   break;
}else if (x > 0){
   printf("Shift %d\n", x);
   stack[++top] = ch;
   st[top] = x;
   if (ch == 'i')
      i = i + 2;
   else
```

```
            i++;
        }else if (x < 0){
            int r = -x;
            printf("Reduce %c -> %s\n", lhs[r], p[r]);
            top -= len[r];
            int k = lhs[r] == 'E' ? 0 : lhs[r] == 'T' ? 1: 2;
            int next = g[st[top]][k];
            if (next == -1){
                printf("Rejected\n");
                break;
            }
            stack[++top] = lhs[r];
            st[top] = next;
        }else{
            printf("String Rejected\n");
            break;
        }
        printf("Stack: ");
        for (int j = 0; j <= top; j++)
            printf("%c%d ", stack[j], st[j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT:

```
Enter string: id*id+id
Shift 5
Stack: $0 i5
Reduce F -> id
Stack: $0 F3
Reduce T -> F
Stack: $0 T2
Shift 7
Stack: $0 T2 *7
Shift 5
Stack: $0 T2 *7 i5
Reduce F -> id
Stack: $0 T2 *7 F10
Reduce T -> T*F
Stack: $0 T2
Reduce E -> T
Stack: $0 E1
Shift 6
Stack: $0 E1 +6
Shift 5
Stack: $0 E1 +6 i5
Reduce F -> id
Stack: $0 E1 +6 F3
Reduce T -> F
Stack: $0 E1 +6 T9
Reduce E -> E+T
Stack: $0 E1
String Accepted
```

5. Write a C program for Stack implementation of SLR parsing for the grammar:

S → AB
A → a / €
B → b / €

Program:

```
#include <stdio.h>

#include <string.h>

int action[6][3] = {

  {3, -3, -3},

  {0, 0, 999},

  {0, 5, -5},

  {0, -2, -2},

  {0, 0, -1},

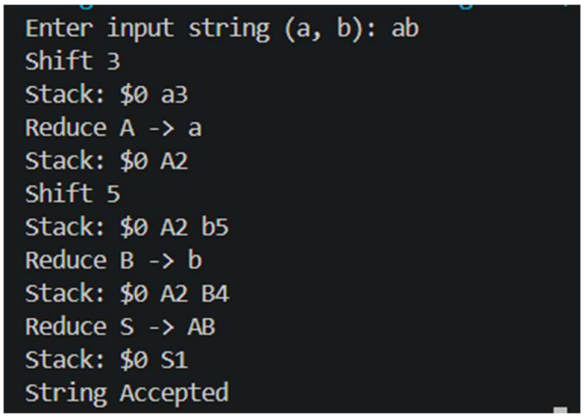
  {0, 0, -4}};
```

```
int go[6][3] = {
   {1, 2, -1},
   {-1, -1, -1},
   {-1, -1, 4},
   {-1, -1, -1},
   {-1, -1, -1},
   {-1, -1, -1}};
char lhs[] = {' ', 'S', 'A', 'A', 'B', 'B'};
int len[] = {0, 2, 1, 0, 1, 0};
char *p[] = {"", "AB", "a", "e", "b", "e"};
int main()
{
   char s[100], stack[100];
   int st[100], top = 0, i = 0,c;
   printf("Enter input string (a, b): ");
   scanf("%99s", s);
   strcat(s, "$");
   stack[0] = '$';
   st[0] = 0;
   while (1){
      char ch = s[i];
      switch (ch){
         case 'a':
            c = 0;
            break;
         case 'b':
            c = 1;
            break;
         case '$':
            c = 2;
```

```
        break;
    default:
        printf("Invalid input\n");
        return 0;
}
int x = action[st[top]][c];
if (x == 999)
{
    printf("String Accepted\n");
    break;
}
if (x > 0)
{
    printf("Shift %d\n", x);
    stack[++top] = s[i];
    st[top] = x;
    i++;
}else if (x < 0){
    int r = -x;
    printf("Reduce %c -> %s\n", lhs[r], p[r]);
    top -= len[r];
    int k = lhs[r] == 'S' ? 0 : lhs[r]=='A'?1:2;
    int next = go[st[top]][k];
    if (next == -1){
        printf("String Rejected\n");
        break;
    }
    stack[++top] = lhs[r];
    st[top] = next;
}else{
```

```
        printf("String Rejected\n");
        break;
      }
      printf("Stack: ");
      for (int j = 0; j <= top; j++)
        printf("%c%d ", stack[j], st[j]);
      printf("\n");
    }
    return 0;
}
```

OUTPUT:



6. Write recursive descent parser for the grammar:

E → TE'
E' → +TE' / €
T → FT'
T' → *FT'/ €
F → (E) / id

Program:

```
#include <stdio.h>
#include <string.h>
char ip[100];
int pos = 0;
int error = 0;
```

```
void E();
void Eprime();
void T();
void Tprime();
void F();
void E()
{
  T();
  Eprime();
}
void Eprime()
{
  if (ip[pos] == '+')
  {
    pos++;
    T();
    Eprime();
  }
}
void T()
{
  F();
  Tprime();
}
void Tprime()
{
  if (ip[pos] == '*')
  {
    pos++;
    F();
```

```
    Tprime();
  }
}
void F()
{
  if (ip[pos] == '(')
  {
    pos++;
    E();

    if (ip[pos] == ')')
      pos++;
    else
      error = 1;
  }
  else if (ip[pos] == 'i' && ip[pos + 1] == 'd')
  {
    pos = pos + 2;
  }
  else
  {
    error = 1;
  }
}
int main()
{
  printf("Enter a string: ");
  scanf("%99s", ip);
  E();
  if (ip[pos] == '\0' && error == 0)
```

```
    printf("String accepted\n");
  else
    printf("Syntax error and string not accepted!\n");
  return 0;
}
```

OUTPUT:

Enter a string: id*id+id

String accepted

7. Write recursive descent parser for the grammar:

S → (L) / a
L → SL'
L' → ,SL' / €

```
#include <stdio.h>
char ip[100];
int pos = 0, error = 0;
void S();
void L();
void Lprime();
void S()
{
  if (ip[pos] == '(')
  {
    pos++;
    L();

    if (ip[pos] == ')')
      pos++;
    else
      error = 1;
```

```
    }
    else if (ip[pos] == 'a')
    {
        pos++;
    }
    else
    {
        error = 1;
    }
}
void L()
{
    S();
    Lprime();
}
void Lprime()
{
    if (ip[pos] == ',')
    {
        pos++;
        S();
        Lprime();
    }
}
int main()
{
    printf("Enter string: ");
    scanf("%99s", ip);
    S();
    if (ip[pos] != '\0')
```

```
    error = 1;


  if (error == 0)
    printf("String is accepted\n");
  else
    printf("String is not accepted\n");
  return 0;
}
```

OUTPUT:

Enter string: (a,a,a)

String is accepted

8. Implement any Static storage allocation strategy.

Program:

```
#include <stdio.h>
int main()
{
  static int a = 10;
  static int b = 20;
  static int c;
  c = a + b;
  printf("Static Storage Allocation\n");
  printf("Value of a = %d\n", a);
  printf("Value of b = %d\n", b);
  printf("Value of c = %d\n", c);
  return 0;
}
```

OUTPUT:

Static Storage Allocation

Value of a = 10

Value of b = 20

Value of c = 30

9. Implement any Stack-based storage allocation strategy.

```
#include <stdio.h>
#include <stdlib.h>
int main() {
  int *stack = NULL;
  int size, top = -1;
  int choice, value;
  printf("Enter the stack size: ");
  scanf("%d", &size);
  if (size <= 0) {
    printf("Invalid stack size!\n");
    return 1;
  }
  while (1) {
    printf("\n1. Push\n");
    printf("2. Pop\n");
    printf("3. Display\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
      case 1:
        printf("Enter the element: ");
        scanf("%d", &value);
        if (top == size - 1) {
          printf("Stack is full!\n");
```

```
        printf("Removing top element: %d\n",
            stack[top]);
        top--;

        if (top == -1) {
          free(stack);
          stack = NULL;
        } else {
          int *temp = realloc(
            stack, top + 1
          );
          if (temp != NULL) {
            stack = temp;
          }
        }
      }
      int *temp = realloc(
        stack, (top + 2) * sizeof(int)
      );
      if (temp == NULL) {
        printf("Memory allocation failed!\n");
        break;
      }
      stack = temp;
      stack[++top] = value;
      printf("%d pushed into the stack.\n",
          value);
      break;
    case 2:
      if (top == -1) {
```

```
            printf("Stack is empty!\n");
        } else {
            printf("Popped element: %d\n",
                stack[top]);
            top--;
            if (top == -1) {
                free(stack);
                stack = NULL;
            } else {
                int *temp = realloc(
                    stack, (top + 1) * sizeof(int)
                );
                if (temp != NULL) {
                    stack = temp;
                }
            }
        }
        break;
    case 3:
        if (top == -1) {
            printf("Stack is empty!\n");
        } else {
            printf("Stack elements are:\n");
            for (int i = top; i >= 0; i--) {
                printf("%d\n", stack[i]);
            }
        }
        break;
    case 4:
        free(stack);
```

```
            printf("Memory released. Exiting...\n");
            return 0;
        default:
            printf("Invalid choice!\n");
    }
  }
  return 0;
}}
```

OUTPUT:

```
PS C:\MinGW\bin> & "C:\Users\Tejasree Nethula\OneDrive\Desktop\cd-lab\stack.exe"
Enter the stack size: 2

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 10
10 pushed into the stack.

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements are:
10
```

❖

10. Implement any Heap-based storage allocation strategy.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define HEAP_SIZE 512
typedef struct Block
{
  int size;
```

```
    int is_free;
    struct Block *next;
} Block;
static char memory_pool[HEAP_SIZE];
static Block *freeList = (Block *)memory_pool;
void init_heap()
{
    freeList->size = HEAP_SIZE - sizeof(Block);
    freeList->is_free = 1;
    freeList->next = NULL;
}
void *heap_alloc(int size)
{
    Block *curr = freeList;

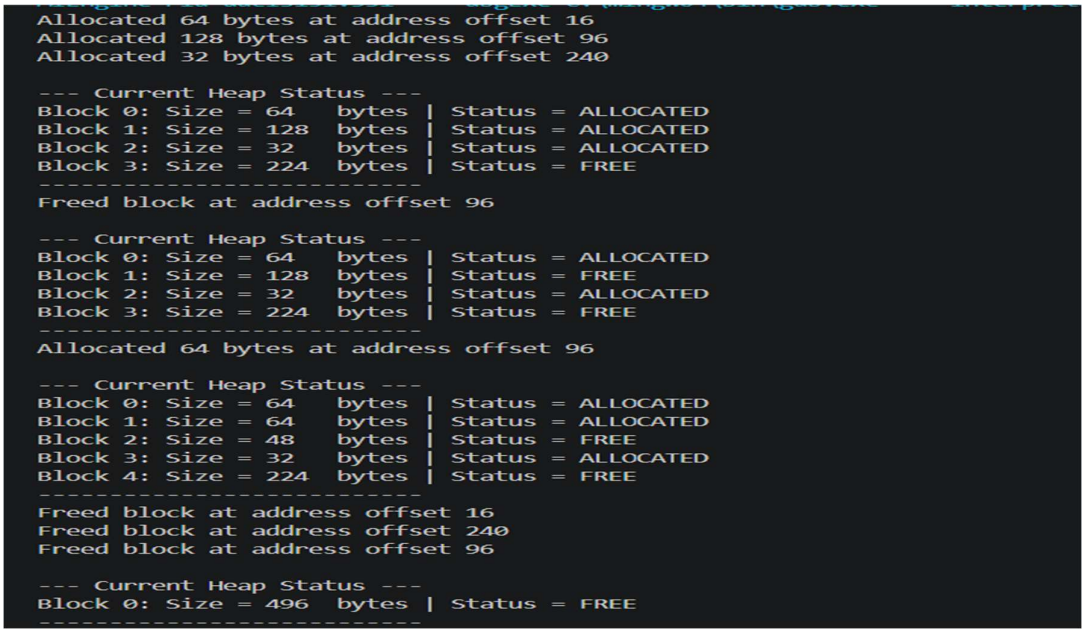
    while (curr != NULL)
    {
        if (curr->is_free && curr->size >= size)
        {
            if (curr->size >= size + (int)sizeof(Block) + 4)
            {
                Block *newBlock = (Block *)((char *)curr + sizeof(Block) + size);
                newBlock->size = curr->size - size - sizeof(Block);
                newBlock->is_free = 1;
                newBlock->next = curr->next;
                curr->size = size;
                curr->next = newBlock;
            }
            curr->is_free = 0;
            printf("Allocated %d bytes at address offset %ld\n",
```

```
            size, (char *)curr + sizeof(Block) - memory_pool);
        return (void *)((char *)curr + sizeof(Block));
      }
      curr = curr->next;
   }
   printf("Error: Heap Out of Memory for %d bytes!\n", size);
   return NULL;
}
void heap_free(void *ptr)
{
   if (!ptr)
      return;
   Block *block = (Block *)((char *)ptr - sizeof(Block));
   block->is_free = 1;
   printf("Freed block at address offset %ld\n", (char *)ptr - memory_pool);
   Block *curr = freeList;
   while (curr && curr->next)
   {
      if (curr->is_free && curr->next->is_free)
      {
         curr->size += sizeof(Block) + curr->next->size;
         curr->next = curr->next->next;
      }
      else
      {
         curr = curr->next;
      }
   }
}
void display_heap_status()
```

```
{
    Block *curr = freeList;
    int index = 0;
    printf("\n--- Current Heap Status ---\n");
    while (curr != NULL)
    {
        printf("Block %d: Size = %-4d bytes | Status = %s\n",
               index++,
               curr->size,
               curr->is_free ? "FREE" : "ALLOCATED");
        curr = curr->next;
    }
    printf("----------------------------\n");
}
int main()
{
    init_heap();
    void *p1 = heap_alloc(64);
    void *p2 = heap_alloc(128);
    void *p3 = heap_alloc(32);
    display_heap_status();
    heap_free(p2); // Free middle chunk to demonstrate fragmentation / free list
    display_heap_status();
    void *p4 = heap_alloc(64); // Fits in the newly freed area
    display_heap_status();
    heap_free(p1);
    heap_free(p3);
    heap_free(p4);
    display_heap_status();
    return 0;
```

}

OUTPUT:



11. Implement Three Address Code using Syntax tree.

Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

struct node {

   char op, name[20], temp[20];

   struct node *l, *r;

};

char s[100];

int p = 0, count = 0;

struct node* create(char op, char *name, struct node *l, struct node *r) {

   struct node *n = calloc(1, sizeof(struct node));

   n->op = op;

   if (name) strcpy(n->name, name);
```

```
    n->l = l;
    n->r = r;
    return n;
}
struct node* expr();
struct node* factor() {
    while (s[p] == ' ') p++;
    if (s[p] == '-') {
        p++;
        return create('~', NULL, factor(), NULL);
    }
    if (s[p] == '(') {
        p++;
        struct node *n = expr();
        p++;
        return n;
    }

    char a[20];
    int i = 0;
    while (isalnum(s[p]) || s[p] == '_')
        a[i++] = s[p++];
    a[i] = '\0';
    return create(0, a, NULL, NULL);
}
struct node* term() {
    struct node *n = factor();
    while (s[p] == '*' || s[p] == '/') {
```

```
      char op = s[p++];
      n = create(op, NULL, n, factor());
   }
   return n;
}
struct node* expr() {
   struct node *n = term();
   while (s[p] == '+' || s[p] == '-') {
      char op = s[p++];
      n = create(op, NULL, n, term());
   }
   return n;
}
char* generate(struct node *n) {
   if (!n->op) return n->name;
   char a[20], b[20], t[20];
   strcpy(a, generate(n->l));
   if (n->op == '~')
      sprintf(t, "t%d", ++count),
      printf("%s = -%s\n", t, a);
   else {
      strcpy(b, generate(n->r));
      sprintf(t, "t%d", ++count);
      printf("%s = %s %c %s\n", t, a, n->op, b);
   }
   strcpy(n->temp, t);
   return n->temp;
}
```

```
int main() {
   char lhs[20];
   printf("Enter expression: ");
   fgets(s, sizeof(s), stdin);
   char *eq = strchr(s, '=');
   if (!eq) return printf("Invalid expression\n"), 0;
   strncpy(lhs, s, eq - s);
   lhs[eq - s] = '\0';
   p = eq - s + 1;
   struct node *root = expr();
   printf("\nThree Address Code:\n");
   printf("%s = %s\n", lhs, generate(root));
   return 0;
}
```

OUTPUT:

```
c:\Users\dooda\OneDrive\Desktop\CD>cd "c:\Users\dooda\OneDrive\Desktop\CD\" && gcc three_.c -o three_ && "c:\Users\dooda\OneDrive\Desktop\CD\"t
hree_
Enter expression: A=(b*-c)*d+(b*-c)*e

Three Address Code:
t1 = -c
t2 = b * t1
t3 = t2 * d
t4 = -c
t5 = b * t4
t6 = t5 * e
t7 = t3 + t6
A = t7
```

12. Implement Three Address Code using Directed Acyclic Graph.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
struct node {
```

```
  char op, name[20], temp[20];
  struct node *l, *r;
};
struct node *nodes[100];
char s[100];
int p = 0, count = 0, countTemp = 0;
struct node* create(char op, char *name, struct node *l, struct node *r) {
  for (int i = 0; i < count; i++) {
    if (nodes[i]->op == op) {
      if (!op && strcmp(nodes[i]->name, name) == 0)
        return nodes[i];
      if (op && nodes[i]->l == l && nodes[i]->r == r)
        return nodes[i];
    }
  }
  struct node *n = calloc(1, sizeof(struct node));
  n->op = op;
  if (name) strcpy(n->name, name);
  n->l = l;
  n->r = r;
  nodes[count++] = n;
  return n;
}
struct node* expr();
struct node* factor() {
  while (s[p] == ' ') p++;
  if (s[p] == '-') {
    p++;
```

```
        return create('~', NULL, factor(), NULL);
    }
    if (s[p] == '(') {
        p++;
        struct node *n = expr();
        p++;
        return n;
    }
    char a[20];
    int i = 0;
    while (isalnum(s[p]) || s[p] == '_')
        a[i++] = s[p++];
    a[i] = '\0';
    return create(0, a, NULL, NULL);
}
struct node* term() {
    struct node *n = factor();
    while (s[p] == '*' || s[p] == '/') {
        char op = s[p++];
        n = create(op, NULL, n, factor());
    }
    return n;
}
struct node* expr() {
    struct node *n = term();
    while (s[p] == '+' || s[p] == '-') {
        char op = s[p++];
        n = create(op, NULL, n, term());
```

```
    }
    return n;
}
char* generate(struct node *n) {
    if (!n->op) return n->name;
    if (n->temp[0]) return n->temp;
    char a[20], b[20], t[20];
    strcpy(a, generate(n->l));
    if (n->op == '~')
        sprintf(t, "t%d", ++countTemp),
        printf("%s = -%s\n", t, a);
    else {
        strcpy(b, generate(n->r));
        sprintf(t, "t%d", ++countTemp);
        printf("%s = %s %c %s\n", t, a, n->op, b);
    }
    strcpy(n->temp, t);
    return n->temp;
}
int main() {
    char lhs[20];
    printf("Enter expression: ");
    fgets(s, sizeof(s), stdin);
    char *eq = strchr(s, '=');
    if (!eq) return printf("Invalid expression\n"), 0;
    strncpy(lhs, s, eq - s);
    lhs[eq - s] = '\0';
    p = eq - s + 1;
```

```
    struct node *root = expr();
    printf("\nThree Address Code:\n");
    printf("%s = %s\n", lhs, generate(root));
    return 0;
}
```

OUTPUT:

```
Enter expression: A=(b*-c)*d+(b*-c)*e

Three Address Code:
t1 = -c
t2 = b * t1
t3 = t2 * d
t4 = t2 * e
t5 = t3 + t4
A = t5
```

13. Write a Lex specification to recognize +ve integers, reals and -ve integers, reals.

```
%{
#include<stdio.h>
%}
%%
        "+"?[0-9]+              {printf("%s:positive integers",yytext);}
        -[0-9]+                 {printf("%s:negative integers",yytext);}
        "-" [0-9]+\.[0-9]+      {printf("%s:negative real numbers",yytext);}
        "+"?[0-9]+\.[0-9]+      {printf("%s:positive real numbers",yytext);}
%%
main()
{
 yylex();
}
```

Compilation: lex noformat.l

cc lex.yy.c -ll

./a.out

24

positive integer

+24.12

positive real number

-24

negative integer

-24.12

negative real number

14. Write a Lex specification for converting real numbers to integers.

```
%{
        int i,j;
        #include<stdio.h>
%}
%%
[0-9]*\.[0-9]+ {
                for(i=0;i<10;i++){
                        if(yytext[i]=='.') exit(0);
                        printf("%c",yytext[i]);
                    }
%%
main()
{
 yylex();
}
```

Compilation: lex realtoint.l

cc lex.yy.c -ll

```
./a.out
24.12
24
```

15. Write a Lex specification to print the number of days in a month using a procedure

```
%{
        #include<stdio.h>
        Int year;
%}
%%
jan|mar|may|july|aug|oct|dec   {printf("31 days");}
april|june|sep|nov    {printf("30 days");}
feb   {leap();}
[a-zA-Z]*     {printf("invalid");}
%%
main()
{
 yylex();
}
leap()
{
        printf(“enter year”);
        scanf(“%d”,&year);
        if(year%4==0)
        printf("29 days");
        else printf("28 days");
}
```

Compilation: lex daysinamonth.l

```
        cc lex.yy.c -ll
        ./a.out
```

jan

31 days

june

30 days

feb

enter year

1984

29 days

16. Write a Lex specification to print a number in between 0-100 in words.

```
%{
        #include<stdio.h>
%}
%%
        2/[0-9] printf("Twenty");
        3/[0-9] printf("Thirty");
        4/[0-9] printf("Forty");
        5/[0-9] printf("Fifty");
        6/[0-9] printf("Sixty");
        7/[0-9] printf("Seventy");
        8/[0-9] printf("Eighty");
        9/[0-9] printf("Ninty");
        10 printf("Ten");
        11 printf("Eleven");
        12 printf("Twelve");
        13 printf("Thirteen");
        15 printf("Fifteen");
        1[4|6|7|8|9] {
                        units();
```

```
                    printf("teen");
                }
[1-9] units();
0  ;
%%
units()
{
 switch(yytext[yyleng-1])
 {
  case'1':printf("One");
          break;
  case'2':printf("Two");
          break;
  case'3':printf("Three");
          break;
  case'4':printf("Four");
           break;
  case'5':printf("Five");
           break;
  case'6':printf("Six");
          break;
  case'7':printf("Seven");
          break;
  case'8':printf("Eight");
          break;
  case'9':printf("Nine");
           break;
}
}
```

Compilation: lex twodigitno.l

cc lex.yy.c -ll

./a.out

6

Six

12

Twelve

24

Twenty Four

17. Write a Lex specification to print a number in between 0-1000 in words.

```
%{
        #include<stdio.h>
%}
%%
        1/[0-9][0-9] print(“One Hundred”);
        2/[0-9][0-9] print(“Two Hundred”);
        3/[0-9][0-9] print(“Three Hundred”);
        4/[0-9][0-9] print(“Four Hundred”);
        5/[0-9][0-9] print(“Five Hundred”);
        6/[0-9][0-9] print(“Six Hundred”);
        7/[0-9][0-9] print(“Seven Hundred”);
        8/[0-9][0-9] print(“Eight Hundred”);
        9/[0-9][0-9] print(“Nine Hundred”);
        2/[0-9] printf("Twenty");
        3/[0-9] printf("Thirty");
        4/[0-9] printf("Forty");
        5/[0-9] printf("Fifty");
        6/[0-9] printf("Sixty");
        7/[0-9] printf("Seventy");
```

```
8/[0-9] printf("Eighty");
9/[0-9] printf("Ninty");
10 printf("Ten");
11 printf("Eleven");
12 printf("Twelve");
13 printf("Thirteen");
15 printf("Fifteen");
1[4|6|7|8|9] {
            units();
                  printf("teen");
               }
[1-9] units();
0     ;
%%
units()
{
 switch(yytext[yyleng-1])
 {
  case'1':printf("One");
           break;
  case'2':printf("Two");
          break;
  case'3':printf("Three");
          break;
  case'4':printf("Four");
           break;
  case'5':printf("Five");
           break;
  case'6':printf("Six");
          break;
```

```
 case'7':printf("Seven");
        break;
 case'8':printf("Eight");
        break;
 case'9':printf("Nine");
         break;
}
}
```

Compilation: lex threedigitno.l

cc lex.yy.c -ll

./a.out

6

Six

12

Twelve

24

Twenty Four

126

One Hundred Twenty Six

242

Two Hundred Forty Two

18. Write a Lex specification to retrieve comments.

```
%{
        #include<stdio.h>
%}
%%
[/][/][a-z A-Z 0-9]*   {printf("%s",yytext);}
[a-z A-Z 0-9]*       {printf(" ");}
```

```
[/][*][a-z A-Z 0-9]*[*][/] {printf("%s",yytext);}
%%
main()
{
 yylex();
}
```

Compilation: lex comments.l

cc lex.yy.c -ll

./a.out

Hello //world

world

hai /*friend*/

friend

19. Write a Lex specification to design a lexical analyzer that recognizes identifiers and keywords of flow control statements of C language.

```
%{
	#include<stdio.h>
%}
%%
If|else|while|do|switch|case|break|for|default	{printf("Keyword");}
IF|ELSE|WHILE|DO|SWITCH|CASE|BREAK|FOR|DEFAULT		{printf("Keyword");}
[A-Za-z_]+[a-zA-Z0-9_]*	{printf("identifier");}
%%
main()
{
 yylex();
}
```

Compilation: lex lexanalysis.l

cc lex.yy.c -ll

./a.out

If

Keyword

FOR

Keyword

Abc123_def

identifier